최기환

Frontend Engineer

반복되는 문제를 구조와 도구로 바꿉니다

rlghks3004@gmail.com GitHub gihwan-dev.tistory.com

DB 성능 모니터링 제품 MaxGauge의 웹 프론트엔드를 만듭니다. 같은 문제가 되풀이되면 공통 구조나 도구로 바꿔서, 다음 사람은 신경 쓰지 않아도 되게 하는 일을 가장 많이 해 왔습니다.

할 수 있는 일

01 **반복되는 구현을 구조로 묶습니다.** 22개 화면에 흩어진 테이블을 컴포넌트 하나로 모았고, 사내 공통 테이블 엔진을 맡아 10,000행 검색 입력 렌더를 `1.9초 → 0.06초`로 줄였습니다.

02 **팀의 암묵지를 도구로 바꿉니다.** 화면 규약과 로컬 환경 구성, 리뷰용 Before/After 영상을 에이전트 스킬로 만들었습니다. 규약 스킬을 넣은 뒤로는 같은 리뷰 지적이 나오지 않습니다.

03 **혼자 맡은 코드도 안심하고 고치게 만듭니다.** 혼자 담당하는 이전 세대 제품군에 최근 2년 MR `250건`을 덮는 회귀 E2E를 붙였습니다.

04 **다른 팀이 매일 쓰는 도구를 만듭니다.** QA팀은 고객 지원에 사내 검색을, 기획팀은 프로토타이핑에 Design Studio를 지금도 씁니다.

경력

주식회사엑셈 · 프론트엔드 개발자 2024.11 - 현재

MaxGauge 6 · MaxGauge 5 · 사내 테이블 엔진 · 사내 AI 도구

MaxGauge 6 · 차세대 DB 성능 분석 웹 제품 2025.02 - 현재

`React · TypeScript · TanStack Query · ECharts`

느린 SQL과 세션의 원인을 찾는 분석 화면을 만들면서, 화면 공통 구조와 팀 개발 환경을 맡고 있습니다.

공통 구조

22개 화면 테이블 통일 화면마다 테이블을 따로 만들어 정렬·필터 동작과 컬럼 이름·단위가 제각각이었습니다. 공통 엔진 위에 제품 규칙만 맡는 어댑터를 두고 컬럼 259개를 레지스트리 하나에 모았습니다. 이제 같은 컬럼은 어느 화면에서나 같은 이름과 동작을 씁니다.

화면 규칙 공통화 차트·테이블 분할 화면 5개가 레이아웃 복원 규칙을 각자 조립하고 있어, 한 곳만 고치면 동작이 어긋날 수 있었습니다. 규칙을 조회 탭 도메인 하나로 옮겼고 SQL·Object·Session 상세 화면의 외곽도 공통 레이아웃으로 묶었습니다.

쓰지 않는 것 걷어내기 소비처 없이 열어 둔 테이블 옵션을 지우고, 화면을 옮기기 전에 공개 인터페이스를 스펙에 필요한 만큼만 남겼습니다. 차트 리사이즈 때 캔버스를 늘려 보여 주고 200ms 뒤 다시 그리던 처리도 글자가 찌그러지는 부작용이 커서 걷어냈습니다.

성능

목록 이동 요청 178건 → 2건 차트에서 SQL을 고르면 목록이 앞 페이지를 전부 받은 뒤에야 그 행으로 갔습니다. 엔진에 아직 받지 않은 구간을 비워 두는 가상 공간을 넣고 선택 행의 페이지만 먼저 받게 했습니다. 실데이터 사례에서 목록 요청은 178건에서 2건으로, 표시까지 걸린 시간은 212.5초에서 11.8초로 줄었습니다.[1]

AI 개발 환경

규약 스킬화 PA 화면 규약이 코드와 개인 메모에 흩어져 있어 같은 위반이 계속 들어왔습니다. 규약 정본을 쓰고, 변경분을 그 규칙으로 검사해 고치는 스킬을 붙였습니다. 그 뒤로 해당 규약에 대한 리뷰 지적은 없어졌습니다.

리뷰 영상 자동화 스크롤이나 드래그처럼 움직임이 핵심인 변경은 리뷰어가 직접 재현해 봐야 판단할 수 있었습니다. 시나리오 파일 하나로 dev와 현재 코드를 같은 조작으로 녹화해 비교 영상을 만드는 스킬을 MR 생성 흐름에 붙였고, 리뷰어는 영상만 보고 차이를 확인합니다.

로컬 환경 스킬 백엔드 서버 6종과 Oracle, 수집 데몬까지 띄워야 해서 로컬 셋업이 어려웠고, 공유 테스트 서버는 다른 본부와 함께 써서 마음대로 쓸 수 없었습니다. 실행 위치와 구성 절차를 나눈 스킬로 노트북이나 사내 Linux에 원하는 백엔드 빌드를 바로 띄우게 했습니다.

CI와 관측

CI 198초 → 94초 유닛 테스트 파일이 5개월 만에 27개에서 341개로 늘며 파이프라인이 무거워졌습니다. 러너부터 늘리지 않고 먼저 재 보니 큐 대기 중앙값은 1.6초였습니다. 잡 실행 방식을 고쳐 MR 파이프라인 중앙값을 198초에서 94초로 줄였습니다.[2]

관측과 린트 Sentry 브라우저 프로파일링을 켜 화면별 프로파일을 받게 했습니다. 디자인 시스템 스타일을 덮어쓴 코드 약 2,540건은 막는 대신, 새로 늘어난 위반만 커밋과 MR에서 보이게 했습니다.

[1] 테스트 서버 실데이터 20,212건에서 같은 점을 골라 전후를 한 번씩 녹화한 기록입니다. [2] 당시 MR의 비교 기록입니다. 변경 전 20회, 후 4회의 중앙값이며 재시도와 캐시 재생성 회차는 뺐습니다.

exem-table · 사내 공통 React 테이블 엔진 2026.03 - 현재

`React · TypeScript · TanStack Table · Vitest · Playwright`

여러 제품이 함께 쓰는 테이블 엔진의 유지보수를 맡고 있습니다.

검색 입력 렌더 1,927ms → 61ms 값 하나만 바뀌어도 테이블 전체가 다시 그려져서, 10,000행에서 검색어를 치면 화면이 2초 가까이 멈췄습니다. TanStack Table v9로 옮기며 행과 셀이 필요한 상태만 구독하게 했습니다. 검색어 14자 입력 렌더는 1,927ms에서 61ms로, 30,000행 힙은 102.7MB에서 27.1MB로 줄었고, 대신 타입체크가 0.9초에서 2.1초로 늘었습니다.[3]

상태 소유와 엔진 경계 값과 콜백을 어떻게 넘기느냐에 따라 "읽기 전용" 같은 중간 상태가 생겨 의도와 다르게 동작했습니다. 제품 파일 142개가 TanStack을 직접 불러서 엔진을 바꾸면 수정이 전체로 번지기도 했습니다. 상태 소유자를 제어와 비제어 둘 중 하나로 정하고 엔진 접근은 파일 하나로 모았습니다. 페이지 조회와 캐시처럼 화면마다 다른 일은 엔진에 넣지 않았습니다.

동작을 검증하는 테스트 설정값이나 mock 반환값을 되읽던 테스트를 실제 브라우저 동작 검증으로 바꿨습니다. 버그는 실패하는 테스트로 먼저 재현하고 나서 고칩니다.

[3] 같은 브라우저에서 v8과 v9를 같은 데이터(10,000행·8컬럼)와 조작으로 돌린 렌더 시간 5회 중앙값입니다. 힙은 GC 후 7회 중앙값입니다.

MaxGauge 5 · 이전 세대 제품군 (Oracle · Tibero · SQL Server) 2025.01 - 현재

`JavaScript · Vue · MSW · Playwright`

금융·공공·제조 고객사 20곳의 요청과 배포 버전을 처리해 왔고, 지금은 혼자 담당합니다.

회귀 E2E 하네스 화면 하나를 확인하려 해도 Java 서버까지 띄워야 했습니다. 실제 DB 스키마를 본뜬 mock DB로 Java 서버 없이 브라우저 E2E를 돌리는 하네스를 만들었고, 최근 2년간 병합된 MR 250건을 모두 회귀 테스트로 덮었습니다.

명령 하나로 로컬 풀스택 라이선스와 인스턴스를 DB에 직접 넣던 기존 셋업은 서버 버전이 조금만 달라도 깨졌습니다. 이 단계를 제품 UI 절차로 바꾸고, `pnpm run dev` 하나로 고객사 배포본 기준 8개 버전 조합을 띄워 8개 시나리오를 모두 통과시켰습니다.

사내 AI 도구 2026.03 - 2026.06

Python · TypeScript · LLM

사내 검색 exemos 이슈·MR·문서·매뉴얼을 함께 찾는 검색 API에서 검색 경로와 운영 배포를 맡았습니다. 지금은 QA팀이 고객 지원에 쓰는 등 사내에서 널리 씁니다.

GitLab 슬래시 커맨드 AI를 쓰려면 개인 PC로 맥락을 옮겨야 했고, 결과를 팀이 함께 보기도 어려웠습니다. MR·이슈 스레드 댓글에서 바로 AI를 불러 함께 검증하고 리뷰하는 봇을 만들었습니다. 운영 오류는 Sentry로, 추론 과정은 Langfuse로 나눠 추적합니다.

Design Studio 기획자가 실제 제품 코드 위에서 화면을 만들고 명세를 쓰는 워크스페이스입니다. 지금도 기획팀이 프로토타이핑에 씁니다.

기술

개발	React · TypeScript · TanStack Table · TanStack Query · Storybook
테스트	Vitest · Playwright · MSW
개발 환경·관측	GitLab CI · Docker · Sentry · Langfuse

학력

동의대학교	정보통신공학과 · 응용 소프트웨어 공학 부전공 | 2017.03 - 2024.03

최근 글

GIHWAN-DEV.TISTORY.COM · 최신 글 5개 · 2026.10.18 확인

- **자본주의 1부 - 돈은 빚이다**
  2026-09-20
- **지금 테스트 코드가 더더욱 중요한 이유**
  2026-09-06
- **스틱 후기**
  2026-08-29
- **Tanstack Table V9 마이그레이션 회고**
  2026-08-26
- **개발자로 살아남기 - 성장에 대한 고민**
  2026-08-09